

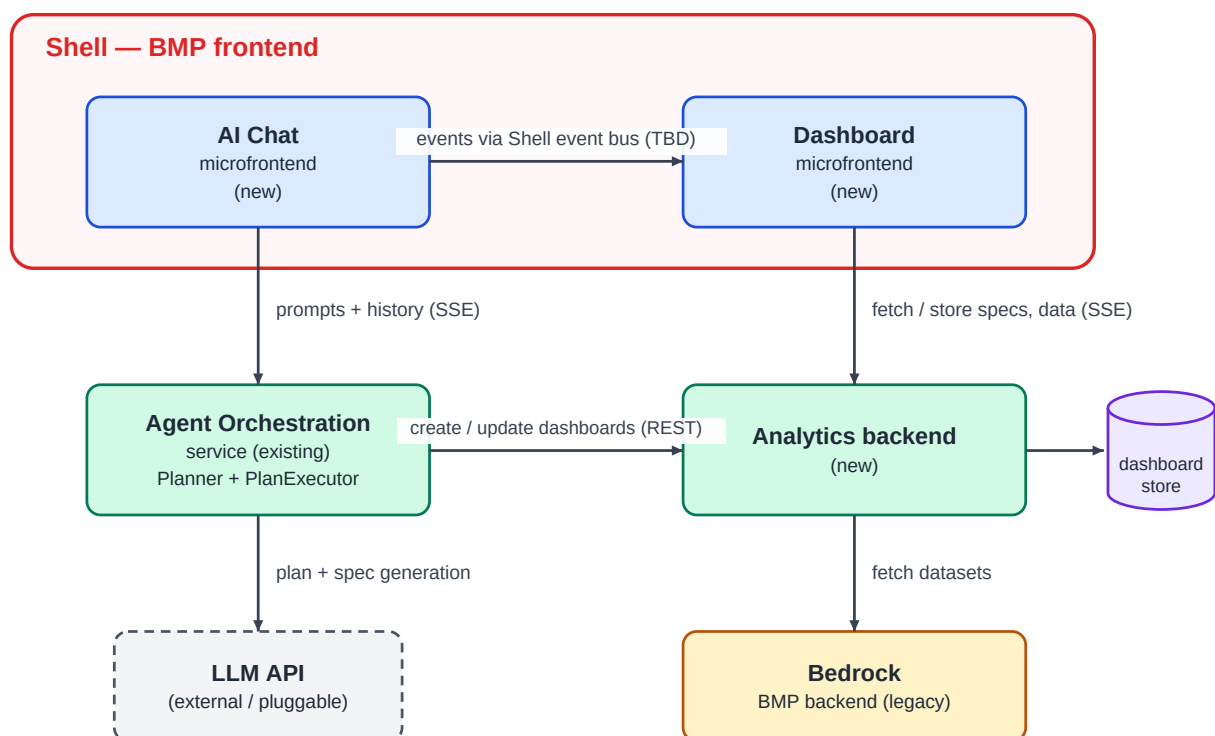
BMP AI Dashboarding — Solution Overview

Architecture and key scenarios, as agreed in the August 2026 workshops. Audience: implementation teams. • 20 Aug 2026

1. Purpose

BMP is getting an AI-assisted dashboarding capability: users describe the dashboard they want in a chat, and the system plans, builds and stores it — while the same dashboards remain viewable and (over time) manually editable in a dedicated dashboard UI. This document describes the services involved, how data flows between them, and the key user scenarios. Items marked **TBD** were not settled in the workshops.

2. Components



Shell (BMP frontend). The main BMP frontend, following a microfrontend architecture. It orchestrates the microfrontends and provides client-side communication between them, likely via an event bus (mechanism **TBD**).

AI Chat microfrontend (new). Chat pane where users send prompts (first use case: building dashboards). Sends each prompt to the Agent Orchestration service together with the previous messages of the current conversation — and, when editing, the relevant dashboard ID. Communicates with its backend over SSE, so it can show progress while a request is being processed.

Dashboard microfrontend (new). Renders dashboards (graphs, KPIs) by fetching dashboard specs from the Analytics backend and mapping them onto components of a chosen charting library. In the long run it will also support building and editing dashboards manually. Communicates with the Analytics backend over SSE.

Agent Orchestration service (existing, extended). Backend that handles user prompts using an LLM; being extended to support the dashboarding domain. It keeps no conversation state server-side — the AI Chat microfrontend sends the history with every request. Internally, a **Planner** asks the LLM to turn the prompt into a plan of actions (chosen from a defined action set), and a **PlanExecutor** executes the

actions one by one. Actions may call the Analytics backend (plain REST) or the LLM (e.g. to generate a dashboard spec). The service stays in control of all side effects; the LLM is used only for what it is good at.

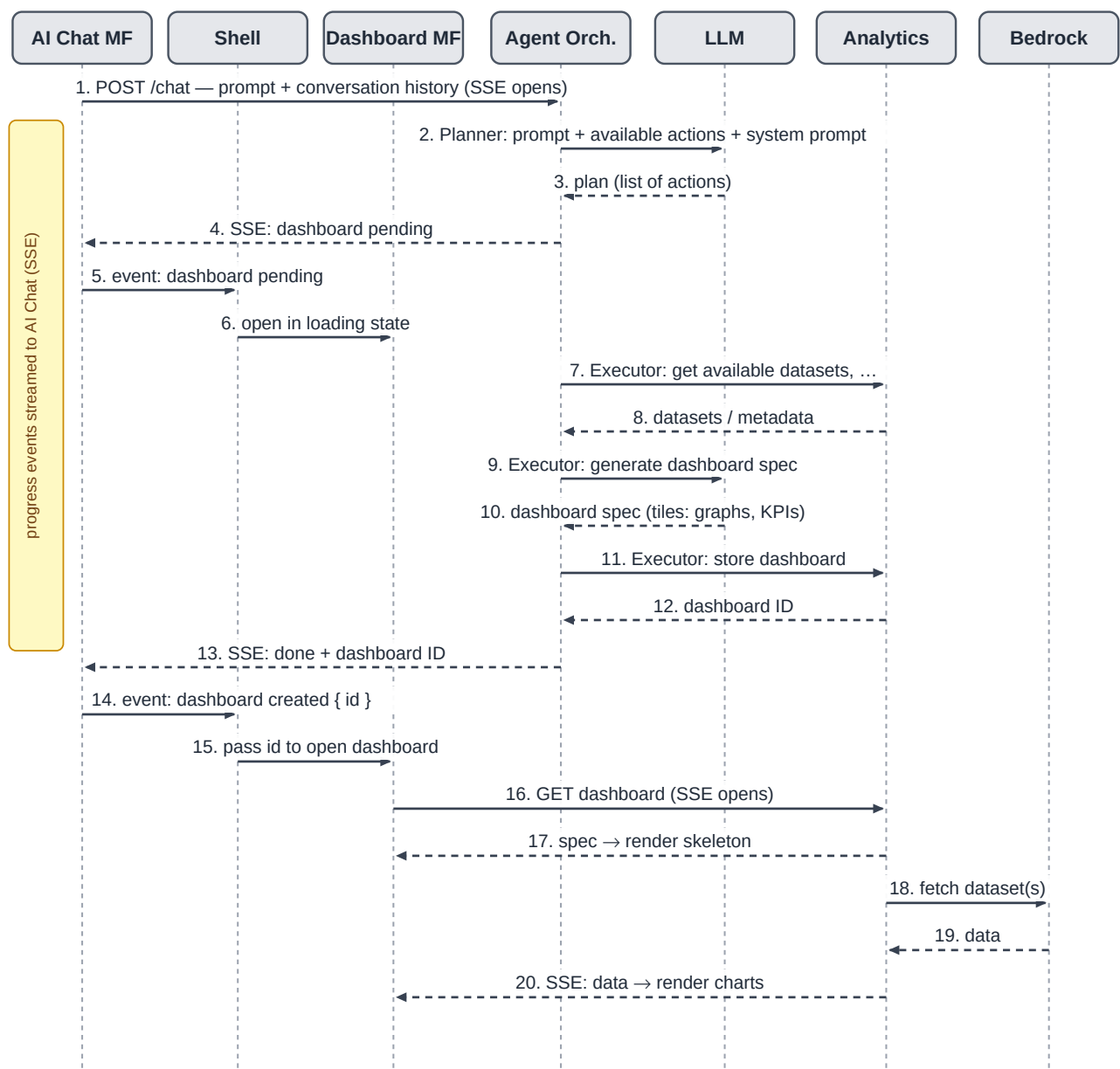
Analytics backend (new). Owns dashboards: stores specs in its own database, serves them to the Dashboard microfrontend, and accepts creates/updates from both the Agent Orchestration service and the Dashboard microfrontend. Also responsible for fetching the data behind the charts from Bedrock.

LLM API (external). An API exposing an LLM — pluggable: Anthropic, OpenAI, a self-hosted model, etc.

Bedrock (BMP backend, legacy). The existing monolith where the data to be charted lives. It has an existing endpoint for fetching pre-defined datasets; an endpoint accepting a dynamic query may not exist yet and might have to be built. Which approach we start with is **TBD** — we may go straight to dynamic queries using a simple hard-coded query.

3. Scenario: create a dashboard via prompt

The user opens the AI chat in the BMP frontend, writes “*I want to build a dashboard of incidents*” and hits send.



1. AI Chat sends the prompt (plus conversation history) to the Agent Orchestration service; the SSE connection stays open and carries progress events throughout.
- 2–3. The Planner sends the prompt, the set of available actions and a system prompt to the LLM, which returns a plan: a list of actions for building the dashboard.
- 4–6. **Early open:** as soon as the plan reveals that a dashboard will be created, the Agent Orchestration service emits a *“dashboard pending”* event on the SSE stream. AI Chat relays it onto the Shell event bus, and the Shell opens the Dashboard microfrontend in a loading state. LLM calls can take a while — this gives the user immediate visual feedback. There is no dashboard ID yet at this point; the loading state is purely a frontend affair.
- 7–10. The PlanExecutor walks the plan. Typical actions: querying the Analytics backend for available datasets and other information, and an LLM call that produces the actual dashboard spec (tiles with graphs or KPIs).
- 11–12. The last action stores the dashboard in the Analytics backend, which responds with the dashboard's ID.
- 13–15. The ID is included in the final SSE response to AI Chat, which emits a thin *“dashboard created”* event (containing the ID) on the Shell event bus. The Shell passes the ID to the already-open Dashboard microfrontend.
- 16–20. The Dashboard microfrontend requests the dashboard from the Analytics backend over SSE. The spec is returned immediately so a skeleton can be rendered; the backend then fetches the data from Bedrock and streams it back to be put into the charts.

Possible later refinement (not in scope now): create the dashboard record in the Analytics backend as the first executor action, so the ID exists up front and can ride along on the “dashboard pending” event — letting the Dashboard microfrontend subscribe to the Analytics backend immediately. This requires create-then-update semantics and handling of placeholder records for failed plans, so we start with the simpler two-event approach above.

4. Scenario: edit a dashboard via chat

1. The user continues the conversation, e.g. *“add a KPI for open incidents”*. AI Chat sends the new prompt, the conversation history and the dashboard's ID to the Agent Orchestration service.
2. The Planner produces a plan as before. The PlanExecutor **fetches the current spec from the Analytics backend** — it never relies on what the conversation remembers, so changes made elsewhere are always respected.
3. An LLM action produces the updated spec, and the executor stores it in the Analytics backend under the same ID. Whether a history of versions is kept (enabling revert) is **TBD**.
4. The response flow to the frontends is the same as in scenario 3 (steps 13–20). The early “dashboard pending” open does not apply here — when editing via chat, the dashboard is typically already open.

5. Scenario: mixing prompt-based and manual editing

The two editing paths are designed to interleave freely:

1. The user creates a dashboard via prompt (scenario 3).
2. The user tweaks it manually in the Dashboard microfrontend, which saves changes directly to the Analytics backend (what exactly triggers a save is **TBD**).
3. The user continues prompting in the chat. Because the Agent Orchestration service always fetches the current spec from the Analytics backend before editing (scenario 4, step 2), the manual changes are preserved.

TBD: how the AI Chat microfrontend learns the dashboard ID when the dashboard was *not* just created in the chat (e.g. the user starts from the Dashboard microfrontend). A candidate — not decided — is to let the Shell event bus carry “active dashboard” context to the AI Chat microfrontend.

6. Manual creation from scratch

For completeness: the Dashboard microfrontend will in the long run also support building dashboards manually from scratch. This is the simple path — the microfrontend edits the spec and stores it via the Analytics backend; the Agent Orchestration service and the LLM are not involved.

7. Technical notes

SSE everywhere between frontend and backend. Both microfrontends talk to their backends over Server-Sent Events, allowing multiple responses per request: progress events from the Agent Orchestration service, and spec-then-data from the Analytics backend.

Deterministic orchestration. The LLM proposes plans and generates specs, but all side effects (reads and writes against the Analytics backend) are plain REST calls made by our code. No LLM tool-calling drives mutations.

Stateless Agent Orchestration. Conversation state lives in the AI Chat microfrontend and is replayed with every request.

Bedrock data access. Bedrock has an existing endpoint for pre-defined datasets. A dynamic query endpoint may not exist yet and might have to be built in Bedrock. The starting point is **TBD**: either begin with pre-defined datasets, or go straight to dynamic queries with a simple hard-coded query — the latter possibly requiring the new Bedrock endpoint first.

Open items (TBD). Event bus mechanism; charting library; save trigger for manual edits; dashboard version history / revert; how AI Chat obtains the active dashboard ID outside the chat-creation flow; whether to start with pre-defined datasets or dynamic queries against Bedrock (the latter may require building a new Bedrock endpoint).